WNDRVR
SUPPORT NETWORK

VXWORKS 7 INSIDE LAYERS TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Inside Layers Tutorial, SR0650

30 June 2020

1. INSIDE LAYERS TUTORIAL

This VxWorks® tutorial provides instructions on how to create, use and share custom layers that reside inside the Wind River installation directory.

VxWorks 7 introduces the layer mechanism as a clean way to encapsulate functionality like software applications, BSPs and Drivers. Layers are added and removed from a VxWorks system without any risk of damage to that system. The layers can reside inside or outside the Wind River installation. This tutorial explores the use of layers inside the Wind River installation.

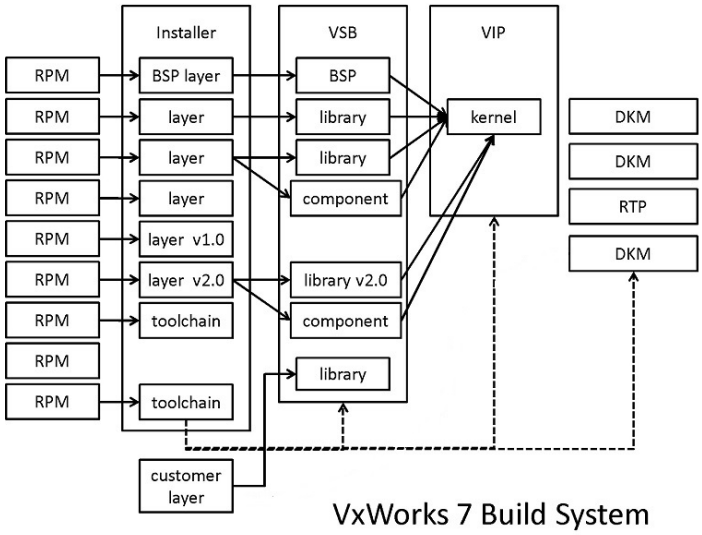
For this tutorial the workflow is as follows:

1. Create a layer.
2. Add the layer to a VSB and test it with a VIP.
3. Export the layer in an RPM.
4. Import the layer RPM.
5. Uninstall the layer.

2. INSIDE AND OUTSIDE LAYERS

VxWorks software components are delivered as RPM files, packaged as layers and may reside inside or outside of the Wind River installation directory.

Source and binary components are presented to the VxWorks build system as RPM files containing layers. A layer encapsulates a piece of functionality that may be built into the VxWorks kernel.



Prior to VxWorks product installation, the functionality within each layer is packaged as an RPM supplied either on installation media, or retrieved from the Wind River servers by the installer or **Product Maintenance** tool. At fresh installation time, the installer takes the RPM files, unpacks them and assembles the initial set of VxWorks layers. The **Product Maintenance** uses the installer to incrementally add new layers into the Wind River installation after a fresh product installation has completed. This is a useful mechanism for both partners and customers who wish to easily add new functionality to VxWorks like a BSP, an API, or a driver.

Layers that exist within the installation directory are located at *installDir* **/vxworks-7/pkgs_v2/**. These layers are referred to as *inside layers*, because they are positioned inside the installation directory structure.

Layers are also a useful way of encapsulating software applications which have been developed by a Wind River customer. However, this customer may not want to be forced to deliver their software application to their own end customers as an internal layer. They may have good reason to decouple their own product installation from the Wind River installation. For this reason, layers may also be positioned outside of the installation. Layers can exist somewhere else on the file system, external to *installDir* **/vxworks-7/pkgs_v2/**. These layers are referred to as *outside layers* because they are located outside of the installation directory.

Outside Layers have all the features and benefits of inside layers except one. The installer cannot install the outside layer to the file system, it can only tell the VxWorks build system that the outside layer is expected to reside there. It is therefore your responsibility to ensure that the RPM file is extracted to the correct directory before the installer reads the RPM and identifies it as an outside layer.

For example tutorials that lead you through the creation of inside and outside layers, please refer to VxWorks 7 Inside Layers Tutorial and VxWorks 7 Outside Layers Tutorial and VxWorks 7 Layer Version Management Tutorial

3. EXAMPLE: CREATING AN INSIDE CUSTOM LAYER AND INTEGRATING IT INTO A VSB

You can create a custom layer within the VxWorks source tree and integrate it into your VxWorks source build (VSB) project using the project build system.

Before You Begin

This example assumes you have access to the **cobble** downloadable kernel module (DKM) example project at *installDir* **\vxworks-7\samples\dkm**.

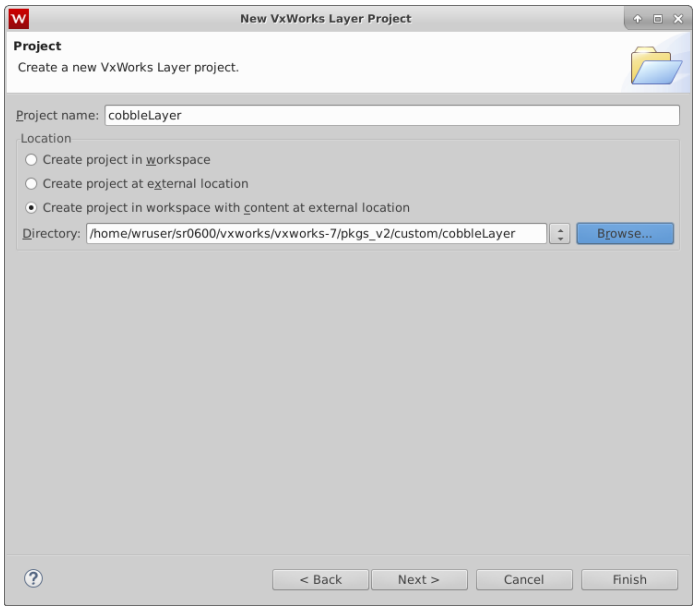
It also assumes you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. Follow the steps below using a VxWorks simulator (VxSim) instance as the target.

About This Task

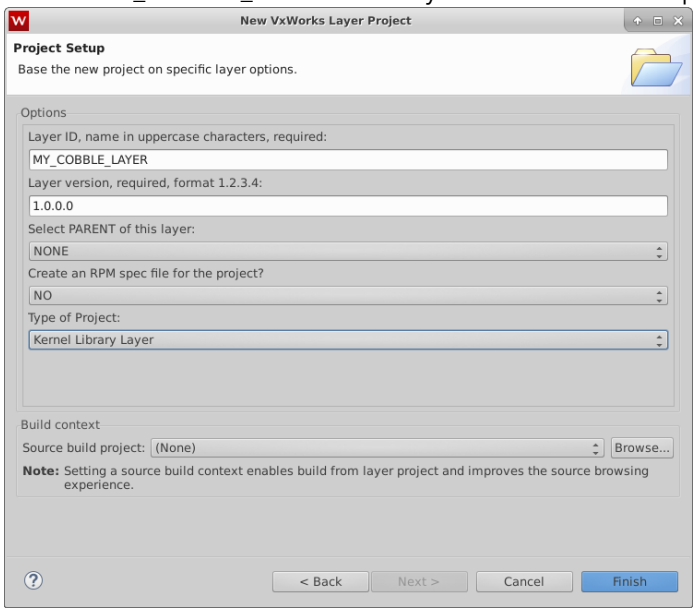
For this tutorial, you use the Workbench **cobble** example DKM project to represent the application code you want to integrate into a layer. Create a layer project, insert the **cobble** DKM application code into it, rebuild the VSB and VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer projects.
    1. Create a root directory within the Wind River installation at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your new custom cobble layer project at *installDir* **\vxworks-7\pkgs_v2\custom\cobbleLayer**.
2. Create a VxWorks layer project in your new layer project folder and call it **cobbleLayer**.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop-down list, select **Layer**. Click **Next**.
    4. Give the Project the name **cobbleLayer**.
    5. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Leave all other options as default and click **Finish**.



3. Populate **cobbleLayer** with **cobble.c**.
    1. Go to *installDir* **\vxworks-7\samples\dkm\cobble**.
    2. Right-click on the **cobble.c** source file and select **Copy**.
    3. In the project explorer, navigate to **cobbleLayer**. Highlight the **src** folder within the **cobbleLayer** project and click **Paste**.
4. Update the **cobbleLayer** metadata file.
    1. Locate the file **layer.vsbl** at the root of the **cobbleLayer** project. Open the file for editing.
    2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Cobble functionality".

```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.0
```

    3. Save the file.

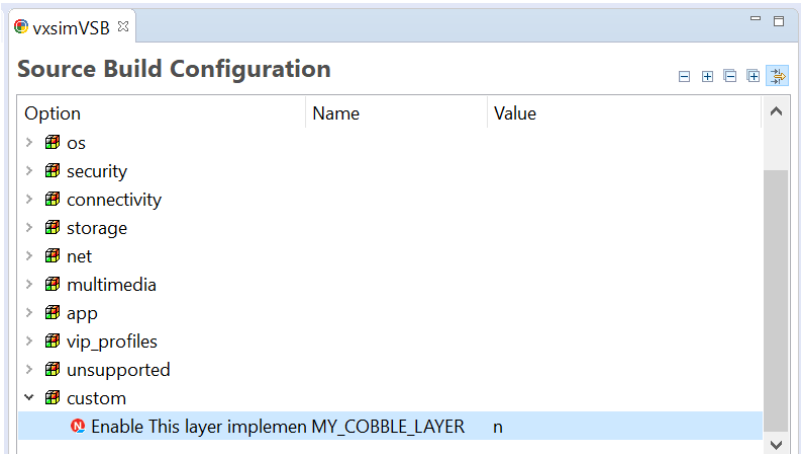
5. Use a host terminal to check that **cobbleLayer**, otherwise known as **MY_COBBLE_LAYER**, does exist.

```
C:\WindRiver\> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
Active workspace is 'C:\WindRiver\workspace'.
WrTool, version 4.1.0
Copyright (c) 2013 - 2014 Wind River Systems, Inc.


C:\WindRiver\> prj vsb info MY_COBBLE_LAYER
Name: MY_COBBLE_LAYER
[...]
[...]
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer
```

6. Verify that your VSB has access to **MY_COBBLE_LAYER** by opening the VSB source build configuration in the project explorer.

   You can see **MY_COBBLE_LAYER** appearing as a component within the Source Build Configuration and the **custom** container.



7. Include **cobbleLayer** in your VSB.
   1. Right-click on the red component within **custom** and select **Add with Dependencies** to add **MY_COBBLE_LAYER** to the VSB.
   2. Observe **MY_COBBLE_LAYER** is now a green component.
   3. In the project explorer, right-click on the VSB and select **Rebuild Project**.
8. Modify your VIP to start the cobble application upon target startup.
   1. Locate the VIP related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open it for editing.
   3. Insert a call to the **cobble.c** progStart( ) function into the **usrAppInit.c** usrAppInit( ) function.

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

9. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

   The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

   - tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
   - tSchlep, which collects the data.
   - tCrunch, which performs calculations on the data to obtain a result value.
   - tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

   Note: If you do not add progStart( ) to **usrAppInit.c**, cobble does not start upon target reboot and **cobbleLayer** does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```

4. EXAMPLE: EXPORTING AN INSIDE CUSTOM LAYER AS AN RPM

VxWorks layer projects are portable, and are shared in the form of an RPM.

Before You Begin

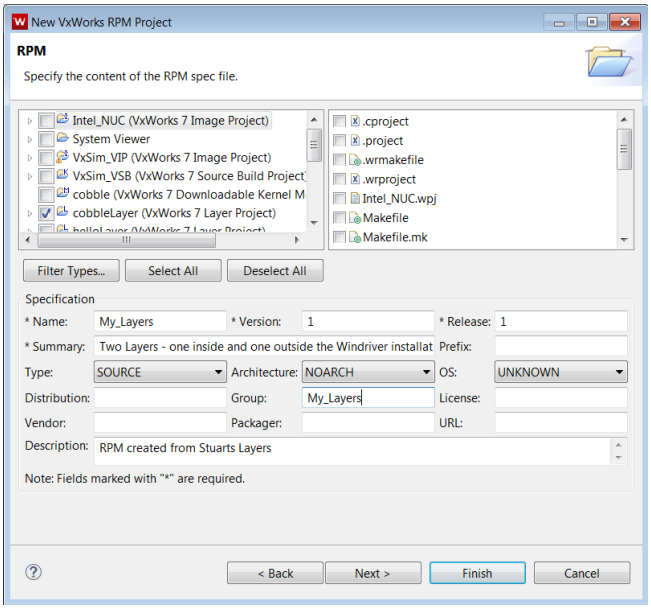
This example assumes you have already completed Example: Creating an Inside Custom Layer and Integrating it into a VSB on page 3. That example results in a VxWorks layer project called **cobbleLayer** that is located internally to the Wind River installation directory and also present in the Workbench project explorer.

About This Task

Now create an RPM file in this example which contains **cobbleLayer**. The generated RPM file is sharable with other VxWorks developers needing to build **cobblelayer** into their VSB.

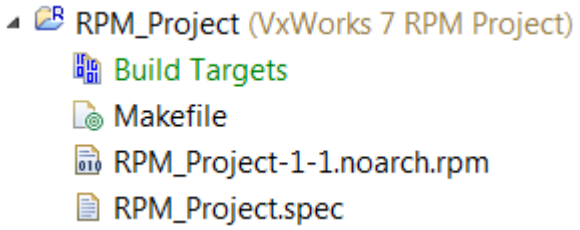
Procedure

1. Create an RPM Project and name it.
    1. From the Workbench menu, select **File > New Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop-down list, select **RPM Project**.
    4. Name the project **RPM_Project** and select **Next**.
2. Give **RPM_Project** a payload.
    1. Select the **cobbleLayer** Project as the RPM payload.
    2. Name the project **My_Layers** and also specify the Group **My_Layers**. Leave the other fields as default.



3. Select **Finish**.
4. Generate the RPM file by selecting right-click on **RPM_Project** in the project explorer and selecting **Build Project**.

The project build results in two files. The file **RPM_Project-1-1.noarch.rpm** is the portable RPM file that may be imported into a fresh Wind River installation on an additional workstation.



5. **cobbleLayer** is internal to the Wind River installation directory and so share **cobbleLayer** by providing **RPM_Project-1-1.noarch.rpm** to another Developer. Sharing this file alone is sufficient to completely export **cobbleLayer**. The Wind River installer that encounters this RPM file will know to extract the layer project correctly into *installDir***\vxworks-7\pkgs_v2\custom\cobbleLayer**.

5. EXAMPLE: IMPORTING AN INSIDE CUSTOM LAYER AS AN RPM

VxWorks layer projects are portable, and when they are shared as RPM files, are easily installed into fresh Wind River Workbench development environments.

Before You Begin

This example assumes you have already completed Example: Exporting an Inside Custom Layer as an RPM on page 7. You must not attempt this example on the workstation that was used with the previous example. This example must be followed on a new, fresh destination workstation containing a fresh installation of Wind River VxWorks 7.

The Workbench project explorer on the destination workstation should contain a VxWorks source build (VSB) project and a VxWorks image project (VIP).

The RPM file to be imported must be copied to the destination workstation as **c:\Wind River\workspace\RPM_Project-1-1.noarch.rpm**.

You must also extract the VxWorks 7 installer into the destination workstation's root Wind River directory. If the installer has already been unzipped into the root Wind River folder, there will be a folder named *installDir* **\installer** and an executeable program named *installDir* **\setupc.exe**.

> Note: The VxWorks 7 installer is not extracted into the destination workstation's root Wind River directory at installation time. Do this manually once installation of the VxWorks 7 product has completed.
>
> The Wind River installer is located on the Wind River server or the source media you used to originally install the product.

About This Task

With this example you will install **cobbleLayer**, previously developed on a source workstation, into a fresh Wind River installation on a destination workstation. You will import and build **cobbleLayer** into the destination workstation's VSB and modify the related VIP so that **cobbleLayer** starts automatically on target reboot.

You will perform all steps below on the destination workstation, not on the original source workstation.

Procedure

1. Open a host terminal and envoke the Wind River installer to install **RPM_Project-1-1.noarch.rpm**.

```
cd c:\WindRiver
setupc.exe -installpath c:\WindRiver -installerUpdateURLs none -yum -y localinstall c:\Wind Riv
er\workspace\RPM_Project-1-1.noarch.rpm
```

> Note: Envoke setupc.exe and not setup.exe. Only the setupc.exe command window will remain open once the installation has completed.

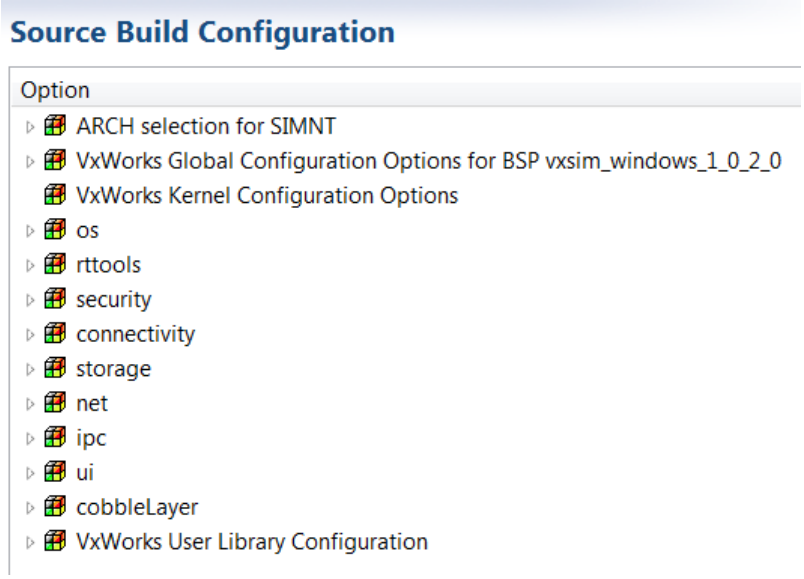
2. Use the same host terminal to verify correct **cobbleLayer** installation.

```
cd c:\WindRiver
C:\WindRiver\> setupc.exe -yum list installed
Default install directory: C:\WindRiver
```

```
Enter install directory or press enter
>
Launching installer...
Searching updates.windriver.com for installer updates.
C:\WindRiver\> ####################
C:\WindRiver\> Installed Packages
C:\WindRiver\> RPM_Layer.noarch  1-1  installed
```

3. Verify that **cobbleLayer** is available to your VSB by opening the VSB source build configuration in the project explorer.

   You can see **cobbleLayer** appearing there as a root component in the Source Build Configuration.



4. Test the **cobbleLayer** you have imported by continuing from Step 7 of Example: Creating an Inside Custom Layer and Integrating it into a VSB on page 3.

6. EXAMPLE: UNINSTALLING AN INSIDE CUSTOM LAYER RPM

VxWorks custom layer projects, when installed as RPM files, can be uninstalled without adversely affecting the Wind River installation, even if the custom layer is installed into the Wind River installation directory.

Before You Begin

This example assumes that you have already completed Example: Importing an Inside Custom Layer as an RPM on page 9.

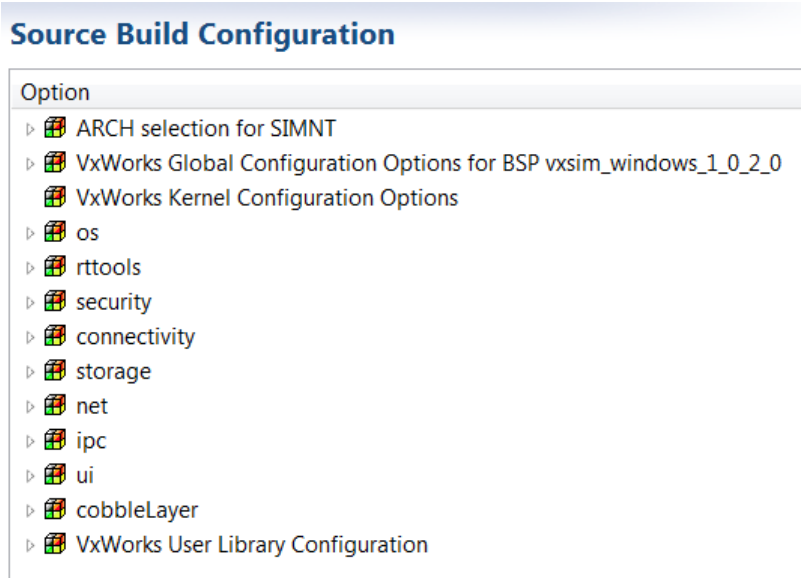
About This Task

A custom layer, **cobbleLayer**, has been installed into Wind River Workbench. You will uninstall **cobbleLayer** in this example.

Procedure

1. Verify that **cobbleLayer** is installed as a custom layer by opening the VSB Source Build Configuration in the project explorer.

   You can see **cobbleLayer** appearing there as a root component in the Source Build Configuration.



2. Uninstall **RPM_Project-1-1.noarch.rpm** from a host shell.

```
cd c:\WindRiver
```

```
setupc.exe -yum remove RPM_Project
```

> Note: Envoke setupc.exe and not setup.exe. Only the setupc.exe command window will remain open once the installation has completed.

3. Verify that **cobbleLayer** is no longer a layer component in Workbench.
   1. In the project explorer, open the VSB Source Build Configuration.

2. Navigate to the bottom of the Source Build Configuration. You can not see **cobbleLayer** appearing as a root component any more.

4. **cobbleLayer** is completely uninstalled.

> Note: Because **cobbleLayer** is an inside layer, the installer has successfully removed the project directory from the file system and also informed Workbench that the custom layer has been removed.

WNDRVR